LEVEL 9 — Library/API Exploration & Language Comparison

How this level works

For each family of functions:

1. Start with **one basic problem**.
2. Solve it without looking at the solution.
3. Identify the library functions that could help.
4. Test each relevant function.
5. Change its arguments.
6. Test normal, boundary and invalid cases.
7. Determine:
    - arguments
    - return value
    - whether it modifies the original object
    - behavior on empty input
    - behavior on invalid input
    - behavior on `NULL` / `None` where relevant
    - behavior with different types
    - exceptions/errors
8. Then solve the **same problem in C, Java and Python**.
9. Compare the three approaches.

9A — String Length

Q55 — Find the length of a string

Given:

Hello World

produce:

Length = 11

Round 1

Do it **without a library function**.

Then investigate:

C

strlen()

Library:

<string.h>

Java

String.length()

Library:

java.lang.String

`String` is automatically available through `java.lang`.

Python

len()

Library:

built-in function

Also investigate:

str.__len__()

Test

For every language test:

""

"a"

"hello"

"Hello World"

" "

"hello\n"

Then investigate Unicode:

"hello"

"नमस्ते"

"😀"

Questions

- Does C count characters or bytes?
- Does Java's `length()` count Unicode code points or UTF-16 code units?
- What does Python's `len()` count?
- Why can these produce different results for the same visible text?

This is our **first major language-difference exercise**.

9B — Copying Strings

Q56 — Copy one string into another

C

Investigate:

strcpy()

strncpy()

Header:

#include <string.h>

Java

Investigate:

String

StringBuilder

String.copyValueOf()

Also investigate:

new String(...)

Python

Investigate:

str()

and understand Python's immutable strings.

Cases

Test:

source = "hello"

source = ""

source = "hello world"

Then deliberately create a destination that is too small in C.

Ask:

> What happens?

Then compare this with Java and Python.

Important investigation

Understand why:

strcpy(destination, source);

is fundamentally different from:

String b = a;

and:

b = a

9C — Comparing Strings

Q57 — Compare two strings

Input:

hello

hello

Output:

Equal

Test:

hello / hello

hello / Hello

abc / abd

abc / ab

"" / ""

C

Investigate:

strcmp()

strncmp()

Header:

<string.h>

Determine exactly what the return value means.

Do **not** assume it returns only `-1`, `0`, `1`.

Test:

strcmp("abc", "abd")

strcmp("abd", "abc")

strcmp("abc", "abc")

Java

Investigate:

equals()

equalsIgnoreCase()

compareTo()

compareToIgnoreCase()

Python

Investigate:

==!=<>casefold()lower()

Comparison question

Why is this:

a == b

not generally equivalent to:

a.equals(b)

And why does Python allow:

a == b

naturally for strings?

And why does C require:

strcmp(a, b)

instead of:

a == b

?

9D — Concatenation

Q58 — Join two strings

Input:

Hello

World

Output:

Hello World

C

Investigate:

strcat()

strncat()

Then investigate what happens when the destination buffer is insufficient.

Java

Investigate:

String.concat()

Then:

StringBuilder.append()

Also investigate:

String.join()

Python

Investigate:

+str.join()

Example:

" ".join(["Hello", "World"])

Extension

Create:

joinWords(words, separator)

Then test:

["I", "love", "C"]

["I", "love", "Java"]

["I", "love", "Python"]

with:

" "

","

"-"

""

Compare

Investigate why repeated string concatenation can have different performance implications in:

- C
- Java
- Python

9E — Searching Inside Strings

Q59 — Find a character

Given:

programming

find the first occurrence of:

g

C

Investigate:

strchr()

strrchr()

Header:

<string.h>

Java

Investigate:

indexOf()

lastIndexOf()

contains()

Python

Investigate:

find()rfind()index()rindex()count()__contains__()

and:

in

Test

Search for:

'g'

'p'

'z'

and test an empty string.

Critical investigation

What is the difference between:

find()

index()

in Python?

What happens if the element isn't found?

Compare this with:

strchr()

indexOf()

9F — Substrings

Q60 — Extract part of a string

Input:

University of Hyderabad

Extract:

University

Hyderabad

C

C doesn't have a direct `substring()` function.

Investigate how to accomplish this using:

strncpy()

memcpy()

Header:

<string.h>

Java

Investigate:

substring()

subSequence()

Test:

substring(beginIndex)

substring(beginIndex, endIndex)

Python

Investigate:

[start:end][start:][:end][start:end:step]

Extension

Reverse a substring.

Example:

abcdefgh

extract:

cdef

and produce:

fedc

Language comparison

This is an important one:

> Why does Python have slicing syntax while C and Java traditionally use function/method calls?

9G — Case Conversion

Q61 — Change case

Input:

Hello World

produce:

HELLO WORLD

hello world

C

Investigate:

toupper()

tolower()

Header:

<ctype.h>

Then investigate:

isalpha()

isdigit()

isalnum()

isspace()

islower()

isupper()

Java

Investigate:

String.toUpperCase()

String.toLowerCase()

Character.toUpperCase()

Character.toLowerCase()

Also investigate the overloads involving:

Locale

Python

Investigate:

upper()lower()capitalize()title()swapcase()casefold()

Important comparison

Why does Python have:

casefold()

when we already have:

lower()

?

When should case-insensitive comparison use `casefold()` rather than simply `lower()`?

9H — Removing Whitespace

Q62 — Clean a string

Input:

" Hello World "

Output:

"Hello World"

C

Investigate:

isspace()

from:

<ctype.h>

You'll need to construct the trimming logic yourself.

Java

Investigate:

trim()

strip()

stripLeading()

stripTrailing()

Python

Investigate:

strip()lstrip()rstrip()

Test

" hello "

"\thello\t"

"\nhello\n"

"hello world"

" "

""

Important question

What's the difference between Java:

trim()

and:

strip()

?

9l — Splitting Strings

Q63 — Split a sentence into words

Input:

I love programming

Output:

I

love

programming

C

There isn't a direct modern equivalent of Java/Python's `split()`.

Investigate:

strtok()

Header:

<string.h>

Then investigate:

strtok_r()

if available on your system.

Java

Investigate:

String.split()

Also investigate:

String.split(regex)

String.split(regex, limit)

Python

Investigate:

str.split()str.rsplit()str.splitlines()

Test:

"one two three".split()"one,two,three".split(",")"one two three".split(" ", 1)

Critical investigation

Compare:

C strtok

Java split

Python split

Questions:

- Does the original string change?
- Does the function return a new collection?
- How is the delimiter specified?
- Does Java use regular expressions?
- What happens with repeated delimiters?
- What happens with empty fields?

9J — Joining

Q64 — Join words

Given:

["I", "love", "programming"]

produce:

I love programming

C

Build your own function.

Java

Investigate:

String.join()

and:

Collectors.joining()

Library:

java.util.stream.Collectors

Python

Investigate:

str.join()

Extension

Try:

separator = " "

separator = ","

separator = "-"

separator = "::"

separator = ""

Then investigate:

Why is Python's syntax:

" ".join(words)

instead of:

words.join(" ")

?

9K — Character Classification

Q65 — Analyze every character

Given:

Hello123!

classify every character as:

letter

digit

whitespace

special

C

Investigate every relevant function from:

<ctype.h>

including:

isalpha()

isdigit()

isalnum()

isspace()

islower()

isupper()

ispunct()

isprint()

iscntrl()

isxdigit()

Don't merely read the documentation.

Create test cases for each.

Java

Investigate:

Character.isLetter()

Character.isDigit()

Character.isLetterOrDigit()

Character.isWhitespace()

Character.isLowerCase()

Character.isUpperCase()

Character.isAlphabetic()

Character.isIdeographic()

Python

Investigate:

str.isalpha()str.isdigit()str.isdecimal()str.isnumeric()str.isalnum()str.isspace()str.islower()str.isupper()str.isprintable()str.isascii()

Important comparison

Test:

"5"

"²"

"Ⅳ"

"四"

"ه"

and see how the three languages classify them.

This is an excellent exercise in understanding **Unicode vs ASCII-oriented APIs**.

9L — Searching & Replacing

Q66 — Replace text

Input:

I like Java. Java is powerful.

Replace every:

Java

with:

Python

C

Investigate:

strstr()

and build your own replacement function.

Java

Investigate:

replace()

replaceFirst()

replaceAll()

Python

Investigate:

replace()

Then:

count()

Critical comparison

Understand the difference between:

replace()

replaceFirst()

replaceAll()

especially the fact that `replaceAll()` uses **regular expressions**.

Then compare this with Python's:

replace()

9M — Formatting

Q67 — Build a formatted message

Given:

name = Samad

age = 24

marks = 87.5

produce:

Name: Samad

Age: 24

Marks: 87.50

C

Investigate:

printf()

sprintf()

snprintf()

Then compare:

%d

%f

%.2f

%s

%c

%x

Java

Investigate:

System.out.printf()

String.format()

formatted()

Python

Investigate:

str.format()f-strings

and formatting specifications:

:.2f:d:x:^10<10>10

Important comparison

Solve the exact same formatting problem in all three languages.

Then answer:

> Which language gives you the most concise formatting syntax?
>
> Which gives you the most explicit control?
>
> What are the differences in type checking and formatting errors?

9N — Conversion / Parsing

Q68 — String → Number

Input:

"12345"

convert it to an integer.

Then:

"12.34"

to floating point.

C

Investigate:

atoi()

atof()

strtol()

strtoul()

strtod()

Header:

<stdlib.h>

Do **not** stop at `atoi()`.

Investigate why `strtol()` is generally more useful for robust parsing.

Java

Investigate:

Integer.parseInt()

Long.parseLong()

Double.parseDouble()

Float.parseFloat()

Also:

Integer.valueOf()

and understand the difference between `parseInt()` and `valueOf()`.

Python

Investigate:

int()float()complex()

Test:

"123"

"-123"

"12.5"

"abc"

""

" 123 "

"0x10"

Then investigate Python's optional arguments:

int("10", 2)int("10", 8)int("10", 10)int("10", 16)

9O — The Big Combined Problem

Q69 — Build a Text Analyzer

Create:

text_analyzer(text)

Given:

"Hello, I am learning C programming. I love programming!"

produce:

Characters:

Words:

Lines:

Letters:

Digits:

Spaces:

Special characters:

Uppercase:

Lowercase:

Most frequent character:

Most frequent word:

Longest word:

Shortest word:

Contains "programming": Yes

Occurrences of "programming":

Rule

You are **not allowed to use one giant function**.

Break it into reusable functions.

For example:

countCharacters()

countWords()

countDigits()

countLetters()

countSpaces()

findLongestWord()

findShortestWord()

findFrequency()

containsWord()

9P — Now Make It Language-Specific

Solve Q69 three times.

C

Primarily investigate:

<string.h>

<ctype.h>

<stdlib.h>

<stdio.h>

Important functions to investigate:

strlen

strcpy

strncpy

strcmp

strncmp

strcat

strncat

strchr

strrchr

strstr

strtok

memcpy

memmove

toupper

tolower

isalpha

isdigit

isalnum

isspace

printf

snprintf

strtol

strtod

Java

Investigate:

java.lang.String

java.lang.Character

java.lang.StringBuilder

java.util.Arrays

java.util.ArrayList

java.util.HashMap

java.util.HashSet

java.util.Collections

java.util.regex.Pattern

java.util.regex.Matcher

java.util.stream.Collectors

Important methods include:

String.length()

charAt()

equals()

equalsIgnoreCase()

compareTo()

substring()

contains()

indexOf()

lastIndexOf()

startsWith()

endsWith()

replace()

replaceFirst()

replaceAll()

split()

strip()

trim()

toUpperCase()

toLowerCase()

isEmpty()

isBlank()

Python

Investigate:

built-ins

str

list

tuple

set

dict

collections

re

string

Important functions/methods:

len()

str()

int()

float()

str.upper()

str.lower()

str.casefold()

str.capitalize()

str.title()

str.swapcase()

str.strip()

str.lstrip()

str.rstrip()

str.split()

str.rsplit()

str.splitlines()

str.join()

str.find()

str.rfind()

str.index()

str.rindex()

str.count()

str.replace()

str.startswith()

str.endswith()

str.isalpha()

str.isdigit()

str.isdecimal()

str.isnumeric()

str.isalnum()

str.isspace()

str.islower()

str.isupper()

re.search()

re.match()

re.fullmatch()

re.findall()

re.finditer()

re.sub()

9Q — API Investigation Challenge

Now **I stop telling you which function to use**.

Q70 — Solve this problem

Given:

" The quick brown fox jumps over 123 lazy dogs. "

produce:

Words: 9

Digits: 3

Letters: 35

Spaces: ...

Uppercase letters: 1

Lowercase letters: ...

Longest word: quick/brown/jumps/...

Contains "fox": Yes

Starts with "The": Yes

Ends with "dogs.": Yes

Your job

For each language:

C: Search the appropriate standard-library headers.

Java: Search the Java API documentation for:

- `String`
- `Character`
- `StringBuilder`
- `Pattern`
- `Matcher`

Python: Search the Python documentation for:

- built-in functions
- `str`
- `re`
- `collections`

But don't just find a function that solves the immediate problem.

For **every function you use**, record:

Function:

Library/module:

Arguments:

Argument types:

Return type:

Does it modify the original?

Possible errors/exceptions:

Empty-input behavior:

Invalid-input behavior:

One alternative:

9R — Same Problem, Three Philosophies

Finally, take one problem:

> **"Find the frequency of every word in a paragraph."**

Implement it three ways.

C

Start with:

arrays

strings

strcmp()

strtok()

Then investigate whether you can implement a better solution using:

struct

and later hashing.

Java

Start with:

String

split()

ArrayList

HashMap

Then investigate:

Map.merge()

getOrDefault()

computeIfAbsent()

Python

Start with:

str.split()

dict

Then investigate:

dict.get()collections.Counter

and compare the approaches.

The key objective of Level 9

By the end of this level, you shouldn't merely know:

> "`strlen()` gives string length."

You should be able to think:

> **"I need to determine the length of this object. What type is it? What library owns that operation? What function/method is available? What arguments does it take? What does it return? What happens for an empty value? Does it modify anything? Is there another function that behaves differently? How does Java/Python handle the same operation?"**

That is the skill we're trying to build.

And this will become especially valuable when we reach **C pointers/function pointers, Java OOD, Python OOD, OS APIs, threads and networking**, because then you'll be expected to explore unfamiliar APIs rather than memorize a predefined list.